RecipeFinder - Revised Project Specification

Student: Yanxue Liu
Course: HCI 5840
Date: October 20, 2026

1. General Description of the Project

RecipeFinder is a Python application that helps users decide what to cook based on ingredients they already have at home.

Sometimes people have ingredients available but do not know what meals they can make. Instead of searching for recipes one by one, users can enter their available ingredients, and the program will recommend recipes based on how closely they match.

The main goal of RecipeFinder is to make recipe searching simple and useful while helping users reduce food waste.

[Revised after Version 1]

The application currently uses a command-line interface (CLI) and a small recipe database stored in a CSV file.

Users can enter multiple ingredients separated by commas. The program processes the input, compares the ingredients with the recipes in the database, calculates match percentages, and displays recommendations sorted from highest to lowest.

The program also identifies missing ingredients and allows users to select a recipe to view basic information.

For Version 2, I plan to complete the recipe details, improve input validation, and allow users to perform another search without restarting the program.

A graphical interface may be considered as an optional feature if time permits.

2. Task Vignettes (User Activity "Flow")

1. Enter Available Ingredients

A user wants to make dinner but does not want to buy more groceries. She checks her kitchen and finds eggs, tomatoes, onions, salt, and oil.

She opens RecipeFinder and enters:

egg, tomato, onion, salt, oil

The program separates the ingredients into a list, removes unnecessary spaces, and converts the text to lowercase.

[Implemented in Version 1]

Technical details:

- Receive ingredient names from user input.
- Separate multiple ingredients into a list.
- Remove extra spaces and convert text to lowercase.
- Use spelling correction to help with incorrectly entered ingredient names.
- Store the processed ingredients for recipe matching.

2. Find Matching Recipes

After the user enters her ingredients, RecipeFinder compares them with the ingredient lists stored in the CSV file.

The program calculates a match percentage for each recipe based on how many required ingredients the user already has.

For example, if a recipe requires four ingredients and the user has three of them, the match percentage is 75%.

The program then displays the recipes sorted by match percentage, from highest to lowest.

Each recommendation includes the recipe name, match percentage, and missing ingredients.

[Implemented in Version 1]

Technical details:

- Read recipe information from a CSV file.
- Compare the user's ingredients with each recipe.
- Calculate the match percentage.
- Identify missing ingredients.
- Store recipe results in a list.
- Sort recipes by match percentage.
- Display the recommendations in a readable format.

3. View a Recipe

The user reviews the recommended recipes and selects one by entering its number.

For example, the user selects Tomato Egg Stir Fry, which has a 100% match.

The program displays the recipe name, match percentage, and missing ingredients.

[Partially Implemented — Revised for Version 2]

Currently, the program displays basic information about the selected recipe.

For Version 2, I plan to update the program to display the complete ingredient list and cooking instructions from the existing CSV file.

Technical details:

Implemented:

- Allow users to select a recipe by number.
- Retrieve the selected recipe from the recommendation list.
- Display the recipe name, match percentage, and missing ingredients.

Planned for Version 2:

- Display the complete ingredient list.
- Display cooking instructions.
- Add input validation for invalid recipe numbers.

4. Search Again

[Revised — Planned for Version 2]

After viewing a recipe, the user may want to search again using different ingredients.

Currently, the user must restart the application to perform another search.

For Version 2, I plan to add an option that allows users to return to the ingredient input step without restarting the program.

Technical details:

- Ask users whether they want to search again.
- Allow users to enter a new ingredient list.
- Repeat the matching and recommendation process.
- Allow users to exit the application.

3. Technical “Flow”

[Revised after Version 1]

The application uses Python and a local CSV file to store recipe information.

The main data includes the user’s ingredient list, recipe names, required ingredients, and matching results.

The current program follows this process:

User Input → Input Processing → Recipe Matching → Match Percentage Calculation → Sort Recommendations → Display Results → Recipe Selection → Display Basic Recipe Information

The main components are:

1. User Input

Receives ingredients entered by the user.

2. Input Processing

Separates the input into individual ingredients, removes unnecessary spaces, converts text to lowercase, and performs spelling correction.

3. Recipe Data

Reads recipe names and ingredient lists from a local CSV file. The CSV file also contains cooking instructions, which will be displayed in Version 2.

4. Recipe Matching

Compares the user’s available ingredients with the ingredients required for each recipe.

5. Match Percentage Calculation

Calculates the percentage of required ingredients available to the user.

6. Recipe Recommendations

Sorts recipes by match percentage and displays the results, including missing ingredients.

7. Recipe Selection

Allows users to select a recipe by number and view basic information.

Planned Technical Improvements for Version 2

The following improvements will be added:

- Input validation to prevent errors when users enter invalid recipe numbers.
- Complete recipe information, including cooking instructions.
- A repeated-search option so users can enter different ingredients without restarting the application.
- Additional testing to improve program reliability.

The application will continue using a local CSV file and a command-line interface.

A graphical interface will remain optional.

4. Version 2 Milestones

[New Section]

Based on my experience developing Version 1, I have revised the project plan to focus on completing the remaining functions and improving the user experience.

Milestone	**Description**	**Priority**
1. Complete Recipe Details	Display the full ingredient list and cooking instructions when users select a recipe.	High
2. Improve User Interaction	Add input validation and allow users to search again without restarting the program.	High
3. Testing and Debugging	Test different ingredient combinations, spelling errors, and invalid inputs.	High
4. Improve the Interface	Explore the possibility of creating a simple graphical interface if time permits.	Optional

My priority is to make sure the command-line version works reliably before adding more advanced features.

5. Final Self-Assessment

[Revised after Version 1]

After developing Version 1, I have a better understanding of how the different parts of RecipeFinder work together.

Initially, I expected recipe matching to be one of the most challenging parts of the project. However, I was able to implement ingredient matching, match percentage calculation, and recipe sorting.

One challenge was organizing the Python code correctly, especially when using loops and conditional statements. I also learned how to store recipe results in a list and display them in a more readable format.

Although the core functions are working, I still need to complete the recipe details and improve how users interact with the application.

For Version 2, I want to focus on making the existing functions more reliable rather than adding too many new features.

I believe this revised plan is realistic for the remaining weeks of the semester.